REDROB HACKATHON

Data & AI Challenge:

Intelligent Candidate Discovery

TEAM

EklaCholo

LEADER

Sakshi Soni

MEMBER

Swikruti Soni

We Read People, Not Documents.

A six-layer candidate intelligence engine that ranks 100K profiles by who they genuinely are — not by how well they wrote their resume.

github.com/thesonisakshi/hired

SOLUTION OVERVIEW

Reading people, not documents

WHAT IS IT

A two-phase intelligent ranking engine

- Offline pre-computation builds deep candidate representations — no time limit
- Online ranking completes in ~16 seconds for 100K candidates on CPU
- Six independent scoring layers, each targeting a different failure mode
- Reasoning column generated from computed scalars — zero hallucination by design

WHAT DIFFERENTIATES IT

Three things no ATS does

Evidence narratives

Embeds what candidates DID (action-verb sentences), not what they claim to know

Variance penalty

High-semantic + low-specificity = untrustworthy. Dimensional disagreement is a red flag.

Behavioral archetypes

7 named hirability patterns — a dormant genius ranks below an active 0.75-fit match

JD UNDERSTANDING & CANDIDATE EVALUATION

Five facets. Not one monolithic query.

30% Core Technical	20% Eval Rigor	25% Product Engineer	10% Mindset	−10% Anti-Patterns
Embeddings, vector DBs, hybrid retrieval, FAISS, reranking	NDCG, MRR, offline benchmarks, A/B testing	Shipped to real users, product co, not pure research	Async writing, ownership, founding-team comfort	Services-only, pure research, LangChain-wrapper only AI

Anti-patterns facet is negatively weighted — high similarity to it PENALISES the candidate score.

RANKING METHODOLOGY

Six layers. One composite score. No keyword matching.

01 JD Facet Decomposition

5 independent semantic query vectors — not one monolith

02 Evidence Narrative

Action-verb sentences only — what they built, not what they listed

03 Specificity Entropy

Regex-based: numbers, versioned tools, failure language, temporal precision

04 Consistency Graph

6 cross-field checks — global contradictions are red flags

05 Honeypot Detection

Timeline logic hard-zeros impossible profiles before scoring

06 Behavioral Archetypes

7 hirability patterns · 0.20–1.30 multiplier · applied last

```
final = (semantic×0.30 + specificity×0.20 + consistency×0.20 + trajectory×0.15 + logistics×0.15 − variance_penalty)
× behavior_multiplier
```

EXPLAINABILITY & DATA VALIDATION

Grounded reasoning. Zero hallucination by design.

How decisions are explained

Every reasoning string is generated from computed scalars — not from LLM inference. The generator references: total years of experience (date arithmetic), company type (lookup table), evidenced skills (regex from descriptions), behavioral archetype (threshold rules), and consistency flags. Every claim is traceable to a specific data field.

How hallucination is prevented

The reasoning generator is a template function over computed scalars — not a language model. It can only reference fields computed from the candidate's actual data. If a skill doesn't appear in the candidate's own descriptions, it cannot appear in the reasoning. Hallucination is architecturally impossible.

How suspicious profiles are handled

Three tiers: Hard disqualification (score = 0) for timeline impossibilities and impossible expert claims. Consistency penalty (up to −60%) for cross-field contradictions. Variance penalty (up to −20%) for profiles that score high on semantics but low on specificity — the 'neat paper' trap.

END-TO-END WORKFLOW

Two phases. One clean pipeline.

OFFLINE PHASE

Run once · No time limit

```
JD → decompose into 5 facets → embed with bge-small-en-v1.5
For each of 100K candidates:
  · Honeypot detection (timeline + expert-claim logic)
  · JD disqualifier check (services-only, no deploy, domain)
  · Build evidence narrative (action verbs only)
  · Specificity entropy score (5 regex dimensions)
  · Cross-field consistency (6 checks + flag list)
  · Career trajectory arc (verb evolution over time)
  · Logistics score (location + notice period)
  · Behavioral archetype (23 signals → 7 classes)
Batch-embed all narratives →
candidate_embeddings.npy
Save all features → features.parquet
```

ONLINE RANKING PHASE

< 5 minutes · CPU only · No network

```
Load artifacts (embeddings + parquet) ..... ~5s
Matrix multiply: 100K × 5 facets ......... ~5s
Weighted facet sum → semantic score ....... ~1s
Composite score (weighted - variance_pen) .. ~2s
Zero-out honeypots + disqualified ......... ~1s
Sort descending → take top 100 ............ <1s
Generate reasoning from scalars ........... ~2s
Write CSV output .......................... <1s

Total: ~16 seconds
```

SYSTEM ARCHITECTURE

Three layers. Each closer to the actual person.

Offline Computation Layer

Any machine · Runs once

- Candidate JSONL.GZ
- Evidence Narrative Builder
- Sentence Encoder (bge-small-en-v1.5)
- Feature Extractors ×6
- Artifact Store (.npy + .parquet)

Online Ranking Layer

CPU only · < 5 min enforced

- Artifact Loader
- Matrix Similarity Scorer
- Weighted Composite Scorer
- Variance Penalty + Behavioral ×
- Top-100 Selector
- Reasoning Generator → CSV

Quality Assurance Layer

Pre-submission validation

- Honeypot Rate Checker (< 10%)
- Format Validator
- Score Monotonicity Checker
- Reasoning Grounding Verifier

RESULTS & PERFORMANCE

Fast, compliant, and defensible.

~16s
Total ranking time for 100K candidates

~0%
Honeypot rate in top 100

<450MB
Peak RAM usage during ranking

0
Network calls during ranking

RUNTIME BREAKDOWN

Step	Time
Load .npy + .parquet artifacts	~5s
Matrix similarity (100K × 5 facets)	~5s
Composite scoring (vectorised)	~3s
Top-100 selection + reasoning	~3s
TOTAL	**~16s**

WHY THIS OUTPERFORMS KEYWORD SYSTEMS

- Surfaces plain-language candidates keyword systems miss
- Suppresses keyword stuffers via specificity entropy
- Eliminates honeypots structurally before ranking
- Accounts for real availability via behavioral archetypes
- Variance penalty punishes 'neat paper' inflated profiles
- Fully CPU-compliant — ~150MB peak for embeddings

TECHNOLOGIES USED

Everything runs locally. No API. No GPU. No surprises.

CORE **sentence-transformers**

Local embedding inference — no API, runs on CPU

MODEL **BAAI/bge-small-en-v1.5**

Best CPU speed/quality ratio for retrieval tasks (33M params, 384-dim)

CORE **numpy**

Vectorised 100K×5 similarity matrix in ~5 seconds on CPU

CORE **pandas + pyarrow**

Fast parquet I/O for pre-computed feature store (~15MB)

UTIL **scikit-learn**

Cosine similarity utilities and data handling

UTIL **regex (stdlib)**

Specificity entropy extraction — 5 dimensions, zero ML overhead

UTIL **python-dateutil**

Robust date parsing for behavioral signal recency scoring

UTIL **gzip (stdlib)**

Native JSONL.GZ streaming without full decompression to disk

Explicitly NOT used: LLMs at inference time · GPU · External APIs · LangChain · Any cloud service during ranking

SUBMISSION ASSETS

Everything needed to run, verify, and reproduce.

`precompute.py`	Offline pipeline — builds all artifacts
`rank.py`	Online ranker — produces submission.csv in ~16s
`helpers/`	6 scoring modules: narrative, specificity, consistency, honeypot, behavioral, trajectory
`requirements.txt`	Pinned dependencies for full reproducibility
`submission.csv`	Final top-100 ranking with grounded reasoning
`submission_metadata.yaml`	Team info, methodology, declarations
`validate_submission.py`	Pre-upload format and honeypot checker

GITHUB REPO

github.com/thesonisakshi/hired

Data & AI Challenge: Intelligent Candidate Discovery — Redrob 2025

REPRODUCE COMMAND

```
python precompute.py \
  --candidates candidates.jsonl.gz

python rank.py \
  --out submission.csv
```

AI Tools Declared: Claude (architecture), Cursor (implementation)

EklaCholo · Sakshi Soni · Swikruti Soni · sakshirs1234@gmail.com

Thank You

We read people, not documents.

github.com/thesonisakshi/hired

EklaCholo · Sakshi Soni · Swikruti Soni